

✓ Android Malware Detection

Importing the Libraries

```
import pandas as pd;
import numpy as np;
import matplotlib.pyplot as plt;
```

Importing the Dataset

```
dataset = pd.read_csv("Malware.csv");
```

```
pd.set_option('display.max_columns', None) # Show all columns
pd.set_option('display.max_rows', None)    # Show all rows if needed
pd.set_option('display.width', 1000)       # Increase display width
pd.set_option('display.max_colwidth', 100) # Increase column width
```

✓ Dataset Preprocessing and Analyzing

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4464 entries, 0 to 4463
Columns: 328 entries, ACCESS_ALL_DOWNLOADS to Label
dtypes: int64(327), object(1)
memory usage: 11.2+ MB
```

```
dataset.shape
```

```
(4464, 328)
```

Dropping Columns with single unique value

```
result = [(i, col) for i, col in enumerate(dataset.columns) if dataset[col].nunique() == 1]

for idx, col in result:
    dataset.drop(col, axis=1, inplace=True)
```

```
len(result)
```

```
43
```

```
dataset.shape
```

```
(4464, 285)
```

```
dataset.duplicated().sum()
```

```
np.int64(1460)
```

Drop Columns with Minority Class < 10%

```
def drop_imbalanced_columns(dataset, threshold=0.10):
    cols_to_drop = []

    for col in dataset.columns:
        vc = dataset[col].value_counts(normalize=True)
        minority_percentage = vc.min()

        if minority_percentage < threshold:
            cols_to_drop.append(col)

    return cols_to_drop
```

```
dropped_cols = drop_imbalanced_columns(dataset, threshold=0.10)
print("Total Dropped columns : ", len(dropped_cols))
```

```
Total Dropped columns : 229
```

```
dataset.shape
```

```
(4464, 56)
```

Dropping Duplicate Rows

```
dataset.drop(columns=dropped_cols,inplace=True)
```

```
dataset = dataset.drop_duplicates(keep="last")
```

```
dataset.shape
```

```
(3004, 56)
```

Splitting DataSet into Independent Variables (x) and Dependent Variables (y)

```
x = dataset.iloc[:, :-1].values;
y = dataset.iloc[:, -1].values;
```

Splitting the Dataset into the Training Set and Test Set

```
from sklearn.model_selection import train_test_split;

x_train , x_test , y_train , y_test = train_test_split(x, y, test_size=0.2, random_state=0);
```

Training the Random Forest on the Training set

```
from sklearn.ensemble import RandomForestClassifier
classifier = RandomForestClassifier(n_estimators = 40, criterion = 'entropy', random_state = 0,max_depth=7);
classifier.fit(x_train, y_train);
```

Evaluating the Individual Model Performance

```
def classification_suitability_Parameters(x,y, x_train, y_train, x_test, y_test, classifier):

    # Methods / Parameters to check Suitability of Classification
    from sklearn.metrics import (
        confusion matrix, accuracy score, precision score, recall score, f1 score,
```

```

roc_auc_score, log_loss, matthews_corrcoef, cohen_kappa_score
)

y_train_predict = classifier.predict(x_train)
y_predict = classifier.predict(x)
y_test_predict = classifier.predict(x_test)

# For classifiers that support predict_proba
if hasattr(classifier, "predict_proba"):
    y_prob = classifier.predict_proba(x)
    y_test_prob = classifier.predict_proba(x_test)
    y_train_prob = classifier.predict_proba(x_train)
else:
    y_prob = None
    y_test_prob = None
    y_train_prob = None

# 1. Confusion Matrix
cm = confusion_matrix(y, y_predict)
cm_train = confusion_matrix(y_train, y_train_predict)
cm_test = confusion_matrix(y_test, y_test_predict)

print("\n\nConfusion Matrix (Whole Set) : \n", cm)
print("Confusion Matrix (Training Set) : \n", cm_train)
print("Confusion Matrix (Test Set) : \n", cm_test)

# 2. Accuracy
print("\nAccuracy (Whole Set) : ", accuracy_score(y, y_predict))
print("Accuracy (Training Set) : ", accuracy_score(y_train, y_train_predict))
print("Accuracy (Test Set) : ", accuracy_score(y_test, y_test_predict))

# 3. Precision, Recall, F1 (macro avg for multi-class)
print("\nPrecision (Whole Set) : ", precision_score(y, y_predict, average="macro"))
print("Precision (Training Set) : ", precision_score(y_train, y_train_predict, average="macro"))
print("Precision (Test Set) : ", precision_score(y_test, y_test_predict, average="macro"))

print("\nRecall (Whole Set) : ", recall_score(y, y_predict, average="macro"))
print("Recall (Training Set) : ", recall_score(y_train, y_train_predict, average="macro"))
print("Recall (Test Set) : ", recall_score(y_test, y_test_predict, average="macro"))

print("\nF1 Score (Whole Set) : ", f1_score(y, y_predict, average="macro"))
print("F1 Score (Training Set) : ", f1_score(y_train, y_train_predict, average="macro"))

```

```

print("F1 Score (Training Set) : ", f1_score(y_train, y_train_predict, average="macro"))
print("F1 Score (Test Set) : ", f1_score(y_test, y_test_predict, average="macro"))

# 4. Specificity (Only valid for binary classification)
if cm.shape == (2, 2):
    tn, fp, fn, tp = cm.ravel()
    tn_tr, fp_tr, fn_tr, tp_tr = cm_train.ravel()
    tn_test, fp_test, fn_test, tp_test = cm_test.ravel()

    print("\nSpecificity (Whole Set) : ", tn / (tn + fp))
    print("Specificity (Training Set) : ", tn_tr / (tn_tr + fp_tr))
    print("Specificity (Test Set) : ", tn_test / (tn_test + fp_test))
else:
    print("\nSpecificity: Not defined for multi-class problems")

# 5. ROC-AUC (Only valid if predict_proba available)
if y_prob is not None:
    if len(set(y)) == 2: # binary
        print("\nROC-AUC (Whole Set) :", roc_auc_score(y, y_prob[:,1]))
        print("ROC-AUC (Training Set) :", roc_auc_score(y_train, y_train_prob[:,1]))
        print("ROC-AUC (Test Set) :", roc_auc_score(y_test, y_test_prob[:,1]))
    else: # multi-class
        print("\nROC-AUC (Whole Set) :", roc_auc_score(y, y_prob, multi_class="ovr"))
        print("ROC-AUC (Training Set) :", roc_auc_score(y_train, y_train_prob, multi_class="ovr"))
        print("ROC-AUC (Test Set) :", roc_auc_score(y_test, y_test_prob, multi_class="ovr"))
else:
    print("\nROC-AUC: Not available (classifier has no predict_proba)")

# 6. Log Loss (if prob available)
if y_prob is not None:
    print("\nLog Loss (Whole Set) :", log_loss(y, y_prob))
    print("Log Loss (Training Set) :", log_loss(y_train, y_train_prob))
    print("Log Loss (Test Set) :", log_loss(y_test, y_test_prob))
else:
    print("\nLog Loss: Not available (classifier has no predict_proba)")

# 7. Matthews Correlation Coefficient (MCC)
print("\nMCC (Whole Set) :", matthews_corrcoef(y, y_predict))
print("MCC (Training Set) :", matthews_corrcoef(y_train, y_train_predict))
print("MCC (Test Set) :", matthews_corrcoef(y_test, y_test_predict))

# 8. Cohen's Kappa

```

```
print("\nCohen's Kappa (Whole Set) :", cohen_kappa_score(y, y_predict))
print("Cohen's Kappa (Training Set) :", cohen_kappa_score(y_train, y_train_predict))
print("Cohen's Kappa (Test Set) :", cohen_kappa_score(y_test, y_test_predict))
```

Evaluating the Hybrid Model Performance

```
classification_suitability_Parameters(x,y, x_train, y_train, x_test, y_test, classifier);
```

Confusion Matrix (Whole Set) :

```
[[1600  65]
 [ 77 1262]]
```

Confusion Matrix (Training Set) :

```
[[1310  53]
 [ 61  979]]
```

Confusion Matrix (Test Set) :

```
[[290  12]
 [ 16 283]]
```

Accuracy (Whole Set) : 0.9527296937416777

Accuracy (Training Set) : 0.9525593008739076

Accuracy (Test Set) : 0.9534109816971714

Precision (Whole Set) : 0.9525510036717342

Precision (Training Set) : 0.9520751700507184

Precision (Test Set) : 0.9535172260994793

Recall (Whole Set) : 0.9517276798830197

Recall (Training Set) : 0.9512306704667306

Recall (Test Set) : 0.9533765974883165

F1 Score (Whole Set) : 0.9521235784709193

F1 Score (Training Set) : 0.9516417739716936

F1 Score (Test Set) : 0.9534046606415028

Specificity (Whole Set) : 0.960960960960961

Specificity (Training Set) : 0.9611151870873074

Specificity (Test Set) : 0.9602649006622517

ROC-AUC (Whole Set) : 0.9878482664890432

ROC-AUC (Training Set) : 0.9878005248603193

ROC-AUC (Test Set) : 0.9878956344548051

Log Loss (Whole Set) : 0.19339407682532986

Log Loss (Training Set) : 0.19098500318989897

Log Loss (Test Set) : 0.20302636292506426

```

MCC (Whole Set) : 0.9042783087465149
MCC (Training Set) : 0.9033054457565213
MCC (Test Set) : 0.906893812684421

Cohen's Kappa (Whole Set) : 0.9042487044771241
Cohen's Kappa (Training Set) : 0.9032846406087155
Cohen's Kappa (Test Set) : 0.9068134496965401

```

Hybrid Model Development

```

from sklearn.ensemble import StackingClassifier
from sklearn.linear_model import LogisticRegression

```

```

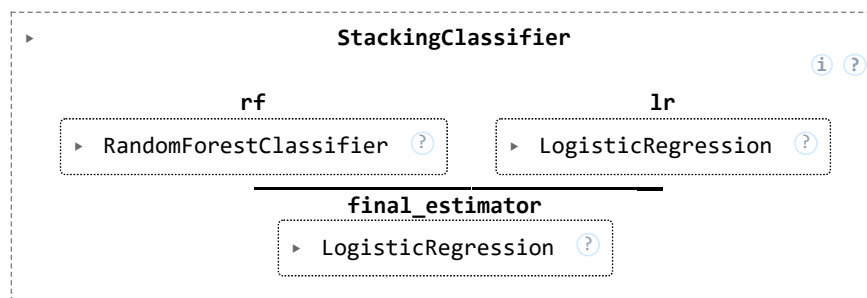
estimators = [
    ("rf", RandomForestClassifier(n_estimators = 40, criterion = 'entropy', random_state = 0,max_depth=7)),
    ("lr", LogisticRegression(max_iter=500))
]

meta_model = LogisticRegression()

stack_model = StackingClassifier(
    estimators=estimators,
    final_estimator=meta_model,
    stack_method='predict_proba',
    passthrough=False
)

```

```
stack_model.fit(x_train, y_train)
```



```
classification_suitability_Parameters(x,y, x_train, y_train, x_test, y_test, classifier);
```

```
Confusion Matrix (Whole Set) :  
[[1600  65]  
 [ 77 1262]]  
Confusion Matrix (Training Set) :  
[[1310  53]  
 [ 61  979]]  
Confusion Matrix (Test Set) :  
[[290  12]  
 [ 16 283]]  
  
Accuracy (Whole Set) : 0.9527296937416777  
Accuracy (Training Set) : 0.9525593008739076  
Accuracy (Test Set) : 0.9534109816971714  
  
Precision (Whole Set) : 0.9525510036717342  
Precision (Training Set) : 0.9520751700507184  
Precision (Test Set) : 0.9535172260994793  
  
Recall (Whole Set) : 0.9517276798830197  
Recall (Training Set) : 0.9512306704667306  
Recall (Test Set) : 0.9533765974883165  
  
F1 Score (Whole Set) : 0.9521235784709193  
F1 Score (Training Set) : 0.9516417739716936  
F1 Score (Test Set) : 0.9534046606415028  
  
Specificity (Whole Set) : 0.960960960960961  
Specificity (Training Set) : 0.9611151870873074  
Specificity (Test Set) : 0.9602649006622517  
  
ROC-AUC (Whole Set) : 0.9878482664890432  
ROC-AUC (Training Set) : 0.9878005248603193  
ROC-AUC (Test Set) : 0.9878956344548051  
  
Log Loss (Whole Set) : 0.19339407682532986  
Log Loss (Training Set) : 0.19098500318989897  
Log Loss (Test Set) : 0.20302636292506426  
  
MCC (Whole Set) : 0.9042783087465149  
MCC (Training Set) : 0.9033054457565213  
MCC (Test Set) : 0.906893812684421  
  
Cohen's Kappa (Whole Set) : 0.9042487044771241  
Cohen's Kappa (Training Set) : 0.9032846406087155  
Cohen's Kappa (Test Set) : 0.9068134496965401
```

